



API Standards



Aakash Vajhallya
Solutions Architect

Introduction

- **Purpose:** To guide you in creating and maintaining robust, consistent, and secure RESTful APIs
- **Scope:** Applicable to all APIs developed within our organization

General Design Principles

- **Consistency:** Follow uniform patterns and naming conventions.
- **Simplicity:** Design APIs to be intuitive and straightforward.
- **Scalability:** Ensure APIs can handle increasing loads efficiently.
- **Security:** Protect data and services with strong security measures.
- **Documentation:** Provide clear and comprehensive documentation for API consumers and fellow developers.

Guiding Design Principles

- **Long-Term Focus:** Design your API with longevity in mind. Prioritize building a solid foundation that can be extended upon rather than rushing to release a version 1.0 product.

- **Spec-Driven Development:** Separate the design and development phases. Use a specification language like RAML to model your API before writing code. This allows for early testing and validation of the design.
- **Versioning as a Last Resort:** Strive to avoid versioning your API whenever possible. Introduce changes seamlessly and maintain backward compatibility. Only version when absolutely necessary due to significant, incompatible changes.
- **User-Centric Design:** Involve potential API users in the design process from day one. Gather feedback through prototypes and iterate on the design to ensure it meets their needs.

Design Standards:

Resource Design:

- Use nouns to represent resources (e.g., /members, /transactions).
- Use hyphens to separate words (e.g., /user-profiles).
- Prefer plural nouns for resource names to indicate collections.
- Support multiple content types (e.g., JSON, XML) to accommodate diverse client preferences.
- Decouple resource design from the underlying architecture to enable independent evolution of the client and server.

Naming Conventions:

- **Endpoint URIs:**
 - Use plural nouns (e.g., /users, /orders).
 - Use hyphens to separate words (e.g., /user-profiles).
 - Avoid using file extensions in URIs.
- **Parameters:**
 - Use camelCase for query parameters (e.g., pageNumber, pageSize).
 - Ensure consistent parameter names across endpoints.

Method Design:

- Use standard HTTP methods (GET, POST, PUT, PATCH, DELETE)

Status Codes:

- **200 OK:** Successful GET, PUT, DELETE.

to perform CRUD operations.

- Clearly define the purpose of each method and its expected behavior.
- Use POST for creating new resources within a collection.
- Use GET for retrieving data.
- Use PUT for replacing an existing resource entirely.
- Use PATCH for updating specific fields of an existing resource.
- Use DELETE for removing a resource.

- **201 Created:** Successful POST.
- **400 Bad Request:** Invalid request.
- **401 Unauthorized:** Authentication required.
- **403 Forbidden:** Access denied.
- **404 Not Found:** Resource not found.
- **500 Internal Server Error:** Server encountered an error.

API Security:

- **Authentication:**
 - Use OAuth 2.0 for secure access.
 - Implement token-based authentication with a smaller expiry window for access tokens.
 - Use SSL for all communications
 - Review and enforce token/credentials storage with all API consumers
- **Authorization:**
 - Define roles and permissions for accessing resources.
 - Use scopes to restrict access to certain actions.
- **Encryption:**
 - **Encryption in Transit:** Always use HTTPS to encrypt data transmitted between the client and server. This protects

API Limiting:

- **Rate Limits:**
 - Define rate limits to prevent abuse (e.g., 100 requests per minute).
- **Limit Headers:**
 - Use headers to inform clients of rate limits (e.g., **X-Rate-Limit-Limit**, **X-Rate-Limit-Remaining**).
- **Throttling:** Adjust the rate at which requests are processed to prevent overloading the API server.
- **Payload Size Limits:**
 - Enforce reasonable payload size limits to prevent excessive resource consumption and potential denial-of-service (DoS) attacks.
 - Clearly communicate payload size limits to API consumers in the documentation.

against eavesdropping and tampering.

- **Encryption at Rest:** Encrypt sensitive data stored in databases or other storage systems. This prevents unauthorized access to data even if the storage is compromised.
- **Payload Encryption:**
 - Sensitive fields in the payload such as passwords or SPII data must have additional level of encryption. Developers can choose asymmetric encryption (For specific fields) or symmetric encryption for entire payload. It is recommended to leverage JWE (See JWE section below).

Versioning:

- **URI Versioning:**
 - Include the version number in the URI (e.g., `/api/v1/users`).
- **Header Versioning:**
 - Use custom headers to specify version (e.g., `X-API-Version: 1`).

Error Handling:

- Provide descriptive error messages that clearly explain the issue and how to resolve it. e.g.:

Bad Error Message	Good Error Messages
Restricted Access	<i>Invalid API Key/Access Token. If you need an API key, you can register for one at http://developers.mydomain.com/register</i>

Permission Denied	<i>Your API Key does not have access to this resource/ or the method attempted on this resource. Please contact our support team if you need access to this resource/ method.</i>
Request Failed	<i>The request was missing required data such as first name, last name email, etc. Additional detail may be found in our documentation by using the link provided below.</i>

- Use standard HTTP status codes to indicate the outcome of requests.
- Consider using established error formats like JSON API or Google Errors that return consolidated list of errors rather than a single error. e.g. JSON API Errors:

```
{
  "error": {
    "errors": [{
      "id": "error1_firstName",
      "code": "XB500",
      "status": "400",
      "title": "User first name cannot be empty",
      "detail": "The first name field is required to have a value",
      "href": "http://docs.domain.ext/users/post"
    },
    {
      "id": "error2_lastName",
      "code": "XB501",
      "status": "400",
      "title": "User last name cannot be empty",
      "detail": "The last name field is required to have a value",
      "href": "http://docs.domain.ext/users/post"
    }
  ]
}
```

JSON API Errors

Pagination:

- Strategies:

Documentation:

- API Specification:

- Use limit and offset parameters (e.g., ?**limit=10&offset=20**).
- Provide links for navigation in response (e.g., **next**, **previous**).

Logging and Monitoring:

- **Logging:** Log all API requests and responses, including timestamps, client IP addresses, and user IDs, for security analysis and troubleshooting.
- **Monitoring:** Monitor API traffic for suspicious activity, such as unusual request patterns or high error rates.

Testing:

- **Automated Testing:**
 - Write unit tests for individual endpoints.
 - Implement integration tests to validate end-to-end scenarios.
- **Mock Servers:**
 - Use mock servers for testing and development.

Quality of Service:

- **Highly available**
 - APIs should be available with no downtime
- **No Business Interruption**

- Use OpenAPI/Swagger for documenting API endpoints.
- Include examples for requests and responses.
- **Developer Guides:**
 - Provide usage examples and best practices.
 - Include tutorials and sample code.

Deprecation Policy:

- **Announcement:**
 - Notify users at least six months before deprecating an API.
- **Support:**
 - Continue supporting deprecated APIs for a reasonable period.
- **Removal:**
 - Provide clear timelines for the removal of deprecated APIs.

- Upgrades to be non-obtrusive; should not create any interruption in business operation
- **Consistent Latency**
 - APIs should respond all the time, and at the right latency as per the business SLA
- **Outage management**
 - APIs should be able to handle unexpected exceptions, outages and resolutions within the defined SLAs

Encryption using JWE:

JSON Web Encryption (JWE) is a standard for securely transmitting encrypted data between parties as a JSON object. It is often used in conjunction with JSON Web Tokens (JWT) to protect sensitive information in API payloads. Here's how you can use JWE to enhance API data protection:

JWE Structure:

A JWE consists of five parts:

1. **Header:** Contains metadata about the encryption algorithm and key used.
2. **Encrypted Key:** The symmetric encryption key used to encrypt the payload, itself encrypted with the recipient's public key.
3. **Initialization Vector (IV):** A random value used to ensure that encrypting the same plaintext with the same key produces different ciphertexts.
4. **Ciphertext:** The encrypted payload.
5. **Authentication Tag:** A value used to verify the integrity of the ciphertext and the authenticity of the sender.

How to Use JWE in API Design:

1. Key Management:

- Establish a secure mechanism for sharing encryption keys between the API provider and authorized consumers. This could involve using asymmetric encryption (e.g., RSA) to exchange symmetric keys (e.g., AES).

- Consider using a key management system (KMS) to store and manage keys securely.

2. Payload Encryption:

- Before sending sensitive data in the API payload, encrypt it using a JWE library or tool.
- Include the necessary JWE headers to specify the encryption algorithm, key, and other relevant parameters.

3. Payload Decryption:

- The API recipient decrypts the JWE payload using their private key and the shared encryption key.
- Verify the authentication tag to ensure the integrity and authenticity of the data.

Benefits of Using JWE:

- **Confidentiality:** Ensures that only authorized parties can access the payload data.
- **Integrity:** Protects against unauthorized modification of the data during transit.
- **Authentication:** Verifies the identity of the sender.
- **Interoperability:** JWE is a widely adopted standard, making it easy to integrate with various systems and platforms.

Considerations:

- **Performance Overhead:** Encryption and decryption can introduce some performance overhead, so consider the trade-off between security and performance.
- **Key Management Complexity:** Secure key management is crucial for the effectiveness of JWE. Choose a robust key management solution that aligns with your security requirements.

